
DATOS LIMPIOS

Tidy data

Fernanda Sobrino

2026

¿Por qué datos limpios?

Los datos nunca llegan listos

- ▶ **Todos** los proyectos de ciencia de datos requieren limpiar los datos primero
- ▶ Los datos de gobierno son el caso extremo: hechos para un trámite, no para un análisis
- ▶ *Data tidying* = darle a la base una **estructura estándar** para poder analizarla
- ▶ No es trabajo previo al análisis: es la parte del análisis donde se cometen los errores caros

Las tres reglas

“**TIDY DATA** is a standard way of mapping the meaning of a dataset to its structure.”

—HADLEY WICKHAM

In tidy data:

- each variable forms a column
- each observation forms a row
- each cell is a single measurement

each column a variable

id	name	color
1	floof	gray
2	max	black
3	cat	orange
4	donut	gray
5	merlin	black
6	panda	calico

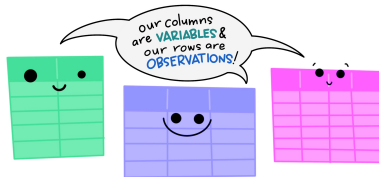
each row an observation

Wickham, H. (2014). Tidy Data. Journal of Statistical Software 59 (10). DOI: 10.18637/jss.v059.i10

Ilustraciones: Allison Horst

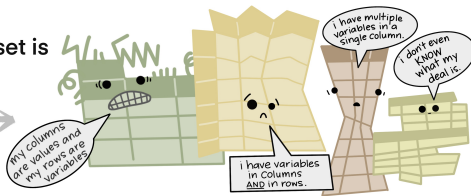
Cada base sucia es sucia a su manera

The standard structure of tidy data means that
"tidy datasets are all alike..."



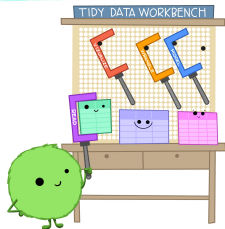
"...but every messy dataset is
messy in its own way."

—HADLEY WICKHAM

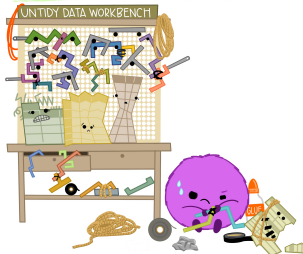


Tidy = las mismas herramientas siempre

When working with tidy data, we can use the **same tools** in **similar ways** for different datasets...



...but working with untidy data often means reinventing the wheel with **one-time approaches** that are **hard to iterate or reuse**.



- ▶ Corolario: si para contestar tu pregunta tienes que “leer” el nombre de una columna, la base no está tidy
- ▶ Y *tidy* depende de la pregunta: la misma tabla puede estar limpia para una cosa y sucia para otra

Cómo trabajaremos: estas slides son el mapa

- ▶ **Estas slides son el resumen conceptual:** cómo se ve cada problema y qué operación lo resuelve
- ▶ **Todo el código vive en el notebook:** `notebooks/Tidy.ipynb` — ahí se corre, se rompe y se verifica
- ▶ El ritmo de la clase: reconocer el problema aquí → al notebook a arreglarlo → de regreso solo para el siguiente concepto
- ▶ Para estudiar: el notebook es la referencia completa; estas slides son el repaso rápido de antes del examen

Los cuatro problemas

El mapa: síntoma, operación, verificación

Síntoma	Operación	Qué verificar después
Datos en el nombre de las columnas	<code>melt</code> (ancho → largo)	renglones nuevos = viejos × columnas apiladas
Una observación en varios renglones	<code>pivot_table</code> (largo → ancho)	que el índice sea llave: si no, promedia solo
Una celda con dos datos	<code>str.split + concat</code>	<code>dtypes</code> : las columnas nuevas nacen como texto
Un dato en varias columnas	concatenar texto	ceros a la izquierda (<code>zfill</code>), tipo final
Los datos en varias tablas	<code>merge</code>	<code>len()</code> antes y después

► Reconocer el síntoma es el 90 % del trabajo; la función es lo fácil

Problema 1: datos en el nombre de las columnas

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

table4

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

country	1999	2000
Afghanistan	19987071	20595360
Brazil	172006362	174504898
China	1272915272	1280428583

table5

country	1999	2000
Afghanistan	19987071	20595360
Brazil	172006362	174504898
China	1272915272	1280428583

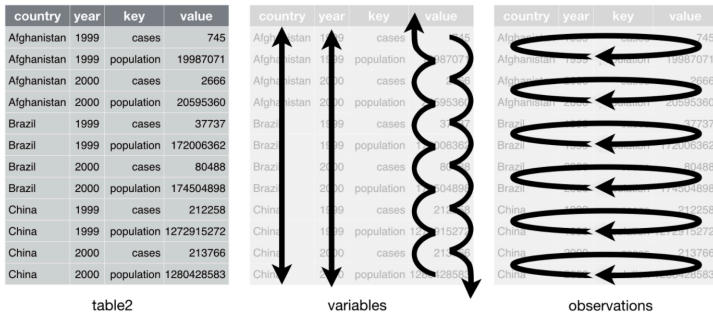
variables

country	1999	2000
Afghanistan	19987071	20595360
Brazil	172006362	174504898
China	1272915272	1280428583

observations

- ▶ 1999 y 2000 no son variables: son **valores** de la variable year
- ▶ No puedes filtrar por año, ni graficar la serie, ni agrupar
- ▶ La solución apila los encabezados en una columna nueva: **de ancho a largo** (melt)

Problema 2: una observación en varios renglones



- ▶ table2 tiene dos renglones por país-año: uno con cases y otro con population
- ▶ Una columna type guarda **el nombre de la variable**, y count guarda su valor
- ▶ No puedes calcular cases / population: están en renglones distintos
- ▶ La solución es la inversa de la anterior: **de largo a ancho** (pivot_table)

Problema 3: una celda con dos datos

country	year	population
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

table3

country	year	population
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

variables

country	year	population
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

values

- ▶ rate contiene "745/19987071": dos variables en una celda
- ▶ Además es **texto**, no número: cualquier promedio va a fallar (o peor, va a concatenar)
- ▶ La solución parte la celda en pedazos y los vuelve columnas (`str.split`)

Problema 4: un dato repartido en varias columnas



country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

country	century	year	rate
Afghanistan	19	99	745 / 19987071
Afghanistan	20	0	2666 / 20595360
Brazil	19	99	37737 / 172006362
Brazil	20	0	80488 / 174504898
China	19	99	212258 / 1272915272
China	20	0	213766 / 1280428583

table6

- ▶ century = 19 y year = 99 son pedazos de un solo dato: 1999
- ▶ Es el problema 3 al revés: aquí hay que **pegar**, no partir
- ▶ Y hay que pegar con cuidado: el año 05 no puede volverse 5

Trampas de la limpieza

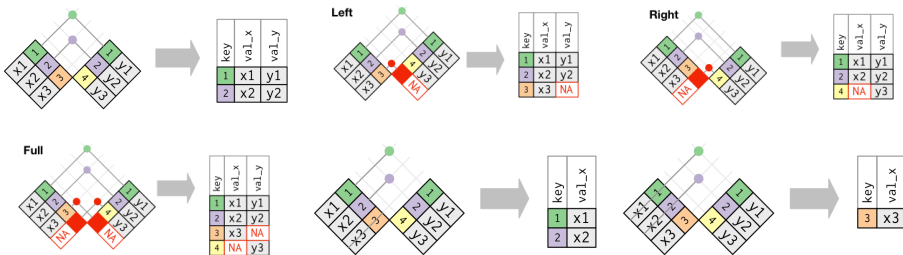
1. **pivot_table agrega en silencio.** Su default es `aggfunc='mean'`: si el índice no es único, promedia y no te dice nada. Si esperas una tabla exacta, verifica primero que el índice sea llave
 2. **Todo lo que sale de texto, nace texto.** Después de partir una celda, las columnas nuevas son `object`. Un “promedio” de texto no truena: concatena
 3. **`str.split` no se queja.** Si un renglón tiene tres pedazos y otro ninguno, rellena con `None` y sigue. Los faltantes nuevos aparecen sin aviso
 4. **Limpiar cambia el conteo de renglones**, y eso es lo que hay que auditar: antes y después de cada operación, cuenta
- Regla transversal: `pandas` no modifica en su lugar — si no asignaste el resultado, no pasó nada

Unir tablas: joins

Los datos viven en varias tablas

- ▶ En `flights` la información está repartida a propósito:
 - ▷ `flights`: un renglón por vuelo
 - ▷ `planes`: un renglón por avión
 - ▷ `weather`: un renglón por fecha-hora
- ▶ **Llave primaria**: identifica de forma única una observación **en su propia tabla**
- ▶ **Llave foránea**: identifica de forma única una observación **en otra tabla**
- ▶ Relaciones posibles: una a una, muchas a una, una a muchas, **muchas a muchas**
- ▶ La mayoría de las bases no te dicen cuál es su llave: hay que encontrarla y **probar** que lo es

Los seis joins, en imagen



Arriba: inner, left, right. Abajo: outer (full), semi, anti.

Resumen: los seis joins

Join	Qué conserva	Para qué
<code>inner</code>	lo que está en ambas	borra datos en silencio
<code>left</code>	todo x, más el match de y	"mi tabla, enriquecida"
<code>right</code>	todo y, más el match de x	un <code>left</code> al revés
<code>outer</code>	todo de ambas	NaN de los dos lados
<code>semi</code>	renglones de x con match	diagnóstico
<code>anti</code>	renglones de x sin match	qué se te va a perder

- ▶ **Usa:** `left` e `inner`. **Con cuidado:** `right` y `outer`. **Para diagnosticar:** `semi` y `anti`
- ▶ `merge` une por **columnas** (default `inner`); `join` por **índices** (default `left`)
- ▶ Escribe siempre `on=` y `how=` — **nunca los dejes al default**

PELIGRO: la llave repetida duplica renglones

- ▶ Si la llave **no es única** en la tabla derecha, merge hace el producto: cada renglón izquierdo se repite una vez por cada match
- ▶ 100 vuelos × 3 registros de clima repetidos para esa hora = **300 renglones**
- ▶ Nadie lanza un error. La tabla simplemente crece
- ▶ Y después: cada promedio, cada suma, cada conteo queda **inflado** — y la conclusión de política pública se construye sobre esa inflación
- ▶ Es el error que buscamos en la **Crítica #1** y en la **Tarea 3**

El chequeo obligatorio: llave única antes, conteo de renglones después, e `indicator=True` para saber cuántos quedaron huérfanos.

Valores faltantes

Explícitos e implícitos

- ▶ **Explícito:** la celda existe y dice NaN — lo ves
- ▶ **Implícito:** la observación **no existe como renglón** — no lo ves
 - ▷ si `stocks` no tiene el 1er trimestre de 2016, ningún `isnull()` te lo va a decir
- ▶ Limpiar la base cambia cuáles son cuáles: `pivot_table` vuelve explícitos los implícitos
- ▶ Para encontrar los implícitos hay una sola receta, y es un join:
 1. Construye la base **completa** con todas las combinaciones del identificador
 2. Únela con la original (`left`) y cuenta la diferencia de renglones
 3. Esa diferencia son las observaciones que nunca supiste que faltaban
- ▶ Pregunta de política pública: ¿la base tiene **todos** los municipios y todos los meses, o solo los que reportaron?

AI notebook

Lo que vamos a hacer en Tidy.ipynb

1. **Los cuatro problemas**, uno por uno, con las tablas de ejemplo: `melt`, `pivot_table`, `str.split + concat`, y concatenar columnas
 2. **Llaves**: encontrar la llave primaria de `planes` y `weather`, y qué hacer cuando no hay
 3. **Los seis joins** sobre tablas chiquitas de goles y asistencias, donde sí se puede contar a mano lo que pasa
 4. **El chequeo de todo** `merge`: `duplicated`, `len()` antes/después, `indicator=True`
 5. **Faltantes explícitos e implícitos** y cómo encontrar los que no se ven
- Los **ejercicios en clase** están marcados en el notebook — se hacen ahí, en el momento

Qué verificar cuando el agente hace esto

- ▶ **Después de cada `melt` o `pivot_table`:** cuenta renglones y celdas. ¿Se perdió información? ¿`pivot_table` promedió en silencio porque el índice no era único?
- ▶ **Después de cada `str.split`:** revisa `dtypes`. Las columnas nuevas nacen como texto, y un “promedio” de texto no truena, concatena
- ▶ **Antes de cada `merge`:** exige la prueba de que la llave es única en la tabla derecha (`duplicated(llave).sum() == 0`)
- ▶ **Después de cada `merge`:** compara `len()` antes y después. Si creció, hay duplicación de renglones y **todos los agregados posteriores están inflados**
- ▶ **Pide el `indicator=True`:** cuántos renglones no hicieron match, y si el agente decidió tirarlos (`inner`) sin decírtelo
- ▶ **Pregunta por los faltantes implícitos:** ¿la base tiene todos los periodos y todas las unidades, o solo las que reportaron?